

Configuration Management Plan — pqc_crypto_module v0.1.0

Disclaimer: This module is prepared for FIPS 140-3 evaluation and is not currently validated.

Standard reference: FIPS 140-3 Section 11 (Life-cycle assurance), ISO/IEC 19790:2012 Section 7.11.

1. Scope

This plan covers the cryptographic module `pqc_crypto_module` (crate root: `crates/pqc_crypto_module/`). It defines the processes for change control, version management, build reproducibility, and artifact integrity throughout the module’s lifecycle.

2. Module Identification

Field	Value
Module name	<code>pqc_crypto_module</code>
Current version	0.1.0
Crate location	<code>crates/pqc_crypto_module/</code>
Boundary	11 source files in <code>src/</code> (see <code>MODULE_SPECIFICATION.md</code> Section 2)
Security level target	FIPS 140-3 Level 1

3. Version Control

3.1 Repository

Property	Value
VCS	Git
Hosting	GitHub (private repository)
Branch model	main is the single production branch
Commit format	Conventional Commits (feat:, fix:, refactor:, docs:, chore:)

3.2 Version Numbering

The module follows [Semantic Versioning](#):

- **MAJOR** — Breaking changes to the public API or cryptographic boundary.
- **MINOR** — New approved algorithms, new API functions, or non-breaking feature additions.
- **PATCH** — Bug fixes, documentation updates, dependency patches with no API change.

The version is recorded in `crates/pqc_crypto_module/Cargo.toml` and is the single source of truth.

3.3 Change Classification

Changes to the module are classified by their impact on the FIPS validation:

Category	Description	Requires re-validation?
Crypto-affecting	Changes to algorithm implementations, self-tests, FSM, key management, or boundary files	Yes
Documentation-only	Updates to FIPS artifacts, comments, or operational guidance	No (notify lab)
Dependency update	Version bump of any direct dependency	Yes (unless patch-only with no algorithm change)
Toolchain update	Rust compiler version change	Yes (rebuild + hash verification required)
Non-boundary	Changes outside <code>crates/pqc_crypto_module/</code>	No

4. Build Environment

4.1 Toolchain

Component	Pinned value	Source
Rust toolchain	nightly-2026-04-20	rust-toolchain.toml (workspace root)
Rust edition	2021	Cargo.toml
Components	rustfmt, clippy	rust-toolchain.toml

4.2 Dependencies

All dependency versions are pinned in two places:

1. **Cargo.toml** — Declares version constraints for direct dependencies.
2. **Cargo.lock** — Pins exact versions for all direct and transitive dependencies.
Committed to the repository.

Direct dependencies (12 crates):

Crate	Version	Purpose
pqcrypto-mlds	0.1.2	ML-DSA-65 (FIPS 204)
pqcrypto-mlkem	0.1.1	ML-KEM-768 (FIPS 203)
pqcrypto-traits	0.3	PQC trait definitions
sha3	0.10	SHA3-256 (FIPS 202)
sha2	0.10	SHA-256 (non-approved, legacy)
hmac	0.12	HMAC-SHA256 (non-approved, legacy)
ed25519-dalek	2.1	Ed25519 (non-approved, legacy)
rand	0.8	RNG interface
rand_core	0.6	OS CSPRNG backend (getrandom)
zeroize	1.7	Memory zeroization (derive feature)
libc	0.2	mlock() for key pinning
thiserror	1.0	Error type derivation

4.3 Build Command

```
cargo clean
RUSTFLAGS="" cargo build --release -p pqc_crypto_module
```

4.4 Artifact Identification

The build artifact is `target/release/libpqc_crypto_module.rlib`.

Each release records:

Field	How to obtain
Artifact SHA-256	<code>sha256sum target/release/libpqc_crypto_module.rlib</code>
Git commit hash	<code>git rev-parse HEAD</code>
Rust compiler version	<code>rustc --version --verbose</code>
Build date (UTC)	<code>date -u +%Y-%m-%dT%H:%M:%SZ</code>
Platform	<code>uname -srm</code>

See `build/reproducible_build.md` for the full deterministic build procedure and verified hash.

5. Change Control Process

5.1 Proposing a Change

1. Create a branch from main.
2. Classify the change per Section 3.3.
3. Implement the change with tests.
4. Run the quality gate:

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test --lib
cargo test --test crypto_boundary
```

5. Open a pull request with:
 - Change classification (crypto-affecting / documentation-only / dependency / toolchain / non-boundary).
 - Description of what changed and why.
 - Test results.

5.2 Review and Approval

- **Crypto-affecting changes** require review by the module owner before merge.
- **Documentation-only changes** may be merged after a single review.
- All changes must pass the CI quality gate before merge.

5.3 Post-Merge Verification

After merging a crypto-affecting change:

1. Perform a clean release build (Section 4.3).
2. Record the new artifact SHA-256.
3. Verify self-tests pass: `cargo test -p pqc_crypto_module`.

4. Update FIPS documentation if the change affects the boundary, FSM, algorithms, or key management.
 5. If the module is under active FIPS validation, notify the lab of the change and its classification.
-

6. Release Process

6.1 Creating a Release

1. Update version in `Cargo.toml`.
2. Update `CHANGELOG.md` with a dated entry.
3. Run full test suite: `cargo test`.
4. Perform clean release build and record artifact hash.
5. Tag the commit: `git tag -a v{VERSION} -m "pqc_crypto_module v{VERSION}"`.
6. Push tag: `git push origin v{VERSION}`.

6.2 Release Record

Each release produces a record containing:

Field	Value
Version	From <code>Cargo.toml</code>
Git tag	<code>v{VERSION}</code>
Git commit	Full SHA
Artifact SHA-256	From build output
Rust toolchain	From <code>rust-toolchain.toml</code>
<code>Cargo.lock</code> SHA-256	<code>sha256sum Cargo.lock</code>
Test results	Pass/fail summary
FIPS classification	Crypto-affecting or not

7. Dependency Management

7.1 Update Policy

- Dependencies are updated only when necessary (security advisory, bug fix, or new feature).
- Every dependency update is treated as a crypto-affecting change.
- After update: rebuild, verify artifact hash, run full test suite.

7.2 Audit

Before any release or lab submission:

```
cargo audit                                # Known CVE scan
cargo tree -d -p pqc_crypto_module         # Duplicate dependency check
cargo tree -p pqc_crypto_module            # Full dependency tree for
review
```

8. Integrity Verification

8.1 Source Integrity

- Git commit history provides full audit trail of every change.
- Cargo.lock committed to repository ensures dependency reproducibility.
- Boundary test (tests/crypto_boundary.rs) verifies no unauthorized crypto imports.

8.2 Build Integrity

- Reproducible builds verified: two independent builds produce identical artifact hashes.
- Build settings documented in build/reproducible_build.md.
- Recommended release profile:

```
[profile.release]
opt-level = 3
lto = true
codegen-units = 1
strip = "symbols"
```

8.3 Runtime Integrity

- Power-up self-tests (4 KATs) execute at module initialization.
 - Any self-test failure transitions the FSM to terminal Error state, blocking all operations.
 - See SELF_TEST_DOCUMENTATION.md for KAT specifications.
-

9. Incident Response

If a vulnerability is discovered in the module or its dependencies:

1. Assess whether the vulnerability affects approved-mode operations.
 2. If yes: develop fix, classify as crypto-affecting, follow change control process.
 3. Rebuild and re-verify artifact hash.
 4. If under active FIPS validation: notify lab immediately with impact assessment.
 5. Document the incident, fix, and verification in CHANGELOG.md.
-

10. Document Control

10.1 FIPS Documentation Artifacts

#	Document	Location
1	Security Policy	SECURITY_POLICY.md
2	Design Document	DESIGN_DOCUMENT.md
3	Module Specification	MODULE_SPECIFICATION.md
4	Finite State Model	FINITE_STATE_MODEL.md
5	Key Management	KEY_MANAGEMENT.md
6	Self-Test Documentation	SELF_TEST_DOCUMENTATION.md
7	Non-Approved Usage	NON_APPROVED_USAGE.md
8	Operational Guidance	OPERATIONAL_GUIDANCE.md
9	Boundary Definition	build/ module_boundary_definition.md
10	Reproducible Build	build/reproducible_build.md
11	Configuration Management	CONFIGURATION_MANAGEMENT.md (this document)

10.2 Update Policy

- FIPS documents are updated whenever a crypto-affecting change modifies the area they cover.
- Each document carries the module version in its header.
- Document changes follow the same change control process as code changes.

11. Audit Trail

The following records constitute the module's audit trail:

Record	Location	Retention
Git commit history	Repository	Permanent
CI/CD test results	CI platform logs	Per CI retention policy
Release records	Git tags + CHANGELOG.md	Permanent
Artifact hashes	Release records	Permanent
Dependency audit results	CI logs + cargo audit output	Per release
Incident reports	CHANGELOG.md entries	Permanent